

Your First Pass

IR surgery with the C++ API

MLIR Summer School — Transformations (1/3)

The arc of this module

#	Session	One-sentence goal
1	Your First Pass	Change IR by hand : navigate, walk, create, replace, erase — inside a real pass.
2	Rewrite Patterns & Dialect Conversion	Express rewrites as patterns , let drivers orchestrate them.
3	The Free Lunch	Canonicalization, folding, CSE, DCE — plug <i>your</i> dialect into the standard cleanup.

The running theme: each session deletes hand-written machinery from the previous one.

- Today we hand-roll everything: traversal, matching, replacement, cleanup.
- Session 2: the *driver* does traversal + bookkeeping; you write only the rewrite.
- Session 3: for many rewrites you don't even write a pass — op hooks do it.

Today's route

1. **Warm-up quiz** — IR anatomy in your own vocabulary
2. **The C++ object model** — `Operation*`, `Region`, `Block`, `Value`
3. **Generic vs. generated ops** — `isa` / `dyn_cast` / `cast`
4. **Use-def chains** — follow the wires, then rewire them
5. `walk()` — traversing IR without writing loops
6. `OpBuilder` — creating new ops at the right place
7. **Worked example** — strength-reduce `mulh` → `shli`
8. **Pass anatomy** — TableGen def → generated base → `runOnOperation`
9. **PassManager & pipelines** — nesting, threading, the CLI
10. **Debugging toolkit** — the flags you'll use every day
11. **Exercise 1** — you build the pass yourself



Quiz: warm-up — point at the pieces

```
func.func @warmup(%a: i32, %n: index) -> i32 {  
  %c2 = arith.constant 2 : i32  
  %r = arith.muli %a, %c2 {answer = 42 : i64} : i32  
  %c0 = arith.constant 0 : index  
  %c1 = arith.constant 1 : index  
  %sum = scf.for %i = %c0 to %n step %c1  
    iter_args(%acc = %r) -> (i32) {  
      %next = arith.addi %acc, %a : i32  
      scf.yield %next : i32  
    }  
  return %sum : i32  
}
```

Point at: ① an operation ② a value ③ a block argument ④ an attribute ⑤ a region

Bonus: how many operations are written here?

✓ Warm-up answers

```
func.func @warmup(%a: i32, %n: index) -> i32 { // op; %a, %n = BLOCK ARGUMENTS
  %c2 = arith.constant 2 : i32                // op; "2 : i32" is an ATTRIBUTE
  %r = arith.muli %a, %c2 {answer = 42} : i32   // op; {answer=42} = attribute
  ...
  %sum = scf.for %i = %c0 to %n step %c1        // op; %sum = OpResult (a Value)
    iter_args(%acc = %r) -> (i32) {           // { ... } = REGION
      %next = arith.addi %acc, %a : i32        // %i, %acc = block arguments!
      scf.yield %next : i32
    }
  return %sum : i32
}
```

- **9 operations** as written (10 with the implicit `builtin.module`).
- Every `%name` is a **Value** — either an op result or a block argument.
- The loop induction variable `%i` and `%acc` are **block arguments** of the region's entry block — not results of `scf.for`.

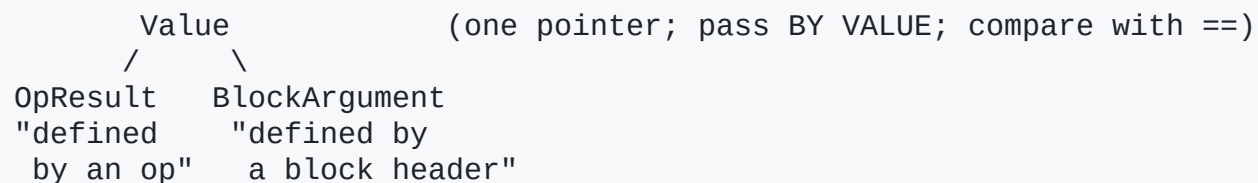
The C++ object model: it's `Operation*` all the way down

At runtime, *every* op — `arith.addi`, `scf.for`, `func.func`, even `builtin.module` — is a `mlir::Operation`:

```
Operation                                (heap object, always handled as Operation*)
├─ name      "arith.muli"
├─ location   Location (where it came from – file:line, or unknown)
├─ operands   [Value, Value, ...]      ← uses of values defined elsewhere
├─ results    [OpResult, ...]          ← the values THIS op defines
├─ attributes {answer = 42 : i64, ...} ← compile-time constant data
└─ regions    [Region, ...]
    Region
    └─ blocks [Block, ...]
        Block
        └─ arguments [BlockArgument, ...]
        └─ operations [Operation, ...] ← recursion!
```

The nesting you know from the textual IR **is** the C++ ownership structure: ops own regions, regions own blocks, blocks own ops.

Value: exactly two kinds



- `OpResult` → `getOwner()` returns the defining `Operation*`, `getResultNumber()`
- `BlockArgument` → `getOwner()` returns the `Block*`, `getArgNumber()`
- `val.getDefiningOp()` — the defining op, or `nullptr` if it's a block argument (function arguments included!)

```
if (Operation *def = operand.getDefiningOp())
    llvm::outs() << "defined by " << def->getName() << "\n";
else // must be a block argument
    llvm::outs() << "block arg #" << cast<BlockArgument>(operand).getArgNumber() << "\n";
```

Handle discipline: `value` has value semantics (copy it freely, compare with `==`). `Operation` is pointer-only. Writing `value*` or copying `operation` — both wrong.

The navigation map

I have...	I want...	Call
Operation *op	its parent	op->getParentOp() , op->getParentOfType<func::FuncOp>() ()
Operation *op	its block / region	op->getBlock() , op->getParentRegion()
Operation *op	what's nested inside	op->getRegions() → region.getBlocks() → block.getOperations()
Block &b	only ops of one type	b.getOps<arith::ConstantOp>()
Operation *op	inputs / outputs	op->getOperand(i) , op->getOperands() , op->getResult(i) , op->getResults()
Operation *	attributes	op->getAttrs() , op->getAttrOfType<IntegerAttr>("...")

Two views of one op: generic vs. generated

The same in-memory `operation` node can be looked at through two lenses:

- **Generic lens** — `operation *`: uniform API, works for *any* op, even unregistered ones.
`getOperand(0)`, `getNumResults()`, `getAttrs()`.
- **Typed lens** — the ODS-generated class, e.g. `arith::AddIOp`: a thin **value-semantic wrapper around the same `operation*`** that adds named accessors.

```
arith::AddIOp add = ...;  
Value lhs = add.getLhs();           // named accessor, from ODS arg name "lhs"  
Value lhs2 = add->getOperand(0);    // identical value, generic spelling  
Operation *raw = add.getOperation(); // unwrap the lens
```

- ODS argument names → accessors: `$lhs` → `getLhs()`, plus `getLhsMutable()` to reassign the operand in place.
- The `->` operator on a typed op forwards to the underlying `operation`, so *everything* from the previous slides works on typed ops too.

When to use which? Generic for op-agnostic code (walking, printing, moving); typed the

Switching lenses: `isa` / `dyn_cast` / `cast`

MLIR uses LLVM's casting infrastructure (not C++ RTTI — MLIR builds with `-fno-rtti`, `dynamic_cast` won't even compile):

```
void inspect(Operation *op) {  
  // Generic API: works for ANY op, registered or not.  
  llvm::outs() << op->getName().getStringRef() << " has "  
    << op->getNumOperands() << " operands, "  
    << op->getNumResults() << " results\n";  
  for (NamedAttribute attr : op->getAttrs())  
    llvm::outs() << "  attr " << attr.getName() << " = " << attr.getValue() << "\n";  
  
  // Typed API: cast to the ODS-generated class for named accessors.  
  if (auto add = dyn_cast<arith::AddIOp>(op)) {  
    Value lhs = add.getLhs();    // == op->getOperand(0)  
    Value res = add.getResult(); // == op->getResult(0)  
  }  
}
```

Call	Behavior
<code>isa<AddIOp>(op)</code>	<code>bool</code> — is it one?
<code>dyn_cast<AddIOp>(op)</code>	typed op, or <i>null op</i> on mismatch — the workhorse

Quiz: this helper shipped upstream

Condensed from a real conversion pass — this ran in production:

```
/// Returns true iff the extension op is fed by a vector.transfer_read.  
static bool isFedByTransferRead(arith::ExtUIOp extOp) {  
    return isa<vector::TransferReadOp>(extOp.getOperand().getDefiningOp());  
}
```

All of the pass's tests passed. Then a user ran it on this function:

```
func.func @f(%v: vector<4xi8>) -> vector<4xi32> {  
    %0 = arith.extui %v : vector<4xi8> to vector<4xi32>  
    return %0 : vector<4xi32>  
}
```

(`vector.transfer_read` = a vector load — its details don't matter here.)

What does the helper do for `%0`'s op?

- ① returns `false` — no `transfer_read` anywhere
- ② returns `true`
- ③ assertion failure / segfault
- ④ verifier error after the pass

✓ It crashes (③) — the null nobody tested 🕒

- `%v` is a **block argument** — `getDefiningOp()` returns `nullptr` (that fine print again).
- `isa<>` on null asserts immediately: `"isa<> used on a null pointer"`. Release builds: undefined behavior.
- Every test fed the `extui` from another op; the first **function argument** in the wild took the whole compiler down.

The real fix is one line — the templated form folds in the null check (`dyn_cast_or_null` under the hood):

```
static bool isFedByTransferRead(arith::ExtUIOp extOp) {  
    // Typed + null-safe in one call:  
    auto read = extOp.getOperand().getDefiningOp<vector::TransferReadOp>();  
    return read != nullptr; // null: block argument OR a different producer  
}
```

Values with no defining op are everywhere: **function arguments**, `scf.for` **induction variables**, `iter_args`. Every backward step through a use-def chain must survive them.

Use-def chains: the wires between ops

SSA in memory: every `Value` keeps a **list of its uses**. A *use* is an `OpOperand` — an edge that knows both endpoints:



```
for (Operation *user : x.getUsers()) // the ops using %x
    llvm::outs() << "used by " << user->getName() << "\n";
for (OpOperand &use : x.getUses()) // the edges: owner + operand index
    llvm::outs() << "operand #" << use.getOperandNumber()
    << " of " << use.getOwner()->getName() << "\n";
```

Cheap queries: `x.use_empty()`, `x.hasOneUse()`, `x.hasNUses(n)`.

Trap: `x.getNumUses()` walks the whole list — *linear time* (documented!). Don't count when you can ask.

Rewiring: `replaceAllUsesWith` and friends

“RAUW” = replace all uses with. Three flavors, one inverse:

```
// 1. Value-level: every use of one SSA value now uses another.
oldVal.replaceAllUsesWith(newVal);

// 2. Conditional: only the uses your predicate accepts.
oldVal.replaceUsesWithIf(newVal, [&](OpOperand &use) {
    return use.getOwner()->getBlock() == someBlock;
});

// 3. Operation-level: all results at once – then the op is dead.
oldOp->replaceAllUsesWith(newOp->getResults());
oldOp->erase(); // safe ONLY once use_empty()

// Inverse direction: rewrite ONE op's own operands.
userOp->replaceUsesOfWith(/*from=*/oldVal, /*to=*/newVal);
```

Two directions — don't mix them up:

- `replaceAllUsesWith` edits *other ops'* operands (everyone pointing at me).
- `replaceUsesOfWith` edits *this op's* operands (what I point at).

Iron rule: RAUW first, `erase()` second. Erasing an op whose results still have uses is a fatal



Quiz: after the RAUW

```
func.func @quiz(%a: i32, %b: i32) -> i32 {  
  %x = arith.addi %a, %b : i32  
  %y = arith.subi %a, %b : i32  
  %u = arith.muli %x, %x : i32  
  %v = arith.addi %u, %y : i32  
  return %v : i32  
}
```

Suppose a pass runs: `x.replaceAllUsesWith(y);` (where `x`, `y` are the C++ values for `%x`, `%y`)

1. Which operations had their operands changed?
2. Can we now safely call `erase()` on the op defining `%x`?
3. Can we safely `erase()` the op defining `%y`?
4. What does `y.getUsers()` yield now?

✓ RAUW quiz answers

```
%x = arith.addi %a, %b : i32    // ← now use_empty(): erasable ✓  
%y = arith.subi %a, %b : i32    // ← now has THREE uses: NOT erasable ✗  
%u = arith.muli %y, %y : i32    // ← the only op that changed (both slots)  
%v = arith.addi %u, %y : i32    // ← unchanged (never used %x)  
return %v : i32
```

1. **Only the `muli` changed** — RAUW rewrites the *uses* of `%x`, nothing else.
2. **Yes** — `%x` is `use_empty()`; `erase()` is safe.
3. **No** — `%y` now has 3 uses; erasing its definer would be the fatal "operation destroyed but still has uses".
4. `y.getUsers()` yields `muli`, `muli`, `addi` — one entry *per use*, so `muli` appears twice.

Traversal, level 0: follow the nesting by hand 🕒

The object model gives you the loops for free (this is the core of the real `-test-print-nesting` pass):

```
void printOperation(Operation *op) {
    llvm::outs() << "op: " << op->getName() << "\n";
    for (Region &region : op->getRegions())
        for (Block &block : region.getBlocks()) // Region is iterable too
            for (Operation &nested : block) // == block.getOperations()
                printOperation(&nested);
}

// Filtered, one nesting level only:
for (arith::ConstantOp c : block.getOps<arith::ConstantOp>())
    llvm::outs() << "constant: " << c.getValue() << "\n";
```

Fine for one level. Tedious for "find every `mul` anywhere in this function" — regions nest arbitrarily deep (`scf.for` inside `scf.if` inside ...).

adapted from `mlir/test/lib/IR/TestPrintNesting.cpp:32-74`

Traversal, level 1: `walk()`

`walk` = “call my callback on every op nested under (and including!) this one”:

```
// Every op, post-order (children before parents), lexical order in blocks:  
root->walk([](Operation *op) { llvm::outs() << op->getName() << "\n"; });  
  
// Typed filtering – the callback's parameter type selects the ops:  
root->walk([](arith::MulIOp op) { /* only muli ops, already typed! */ });  
  
// Parents before children instead:  
root->walk<WalkOrder::PreOrder>([](Operation *op) { /* ... */ });
```

- Default order is **post-order** — by the time you see an op, its regions are done.
- The typed-callback filter is not a cast — it *silently skips* every non-matching op. Interfaces work too: `walk([](LoopLikeOpInterface loop) {...})`.
- `Block::walk` and `Region::walk` exist as well.

Controlling a walk: WalkResult

Return WalkResult from the callback to steer the traversal — advance continues, interrupt aborts everything, skip prunes the subtree:

```
WalkResult result = root->walk([](Operation *op) {  
    if (isa<gpu::LaunchOp>(op))  
        return WalkResult::interrupt(); // abort the entire walk  
    if (isa<scf::ForOp>(op))  
        return WalkResult::skip();      // don't descend into loops  
    return WalkResult::advance();        // keep going (the default)  
});  
if (result.wasInterrupted()) { /* found one */ }
```

- A void callback = always advance.
- WalkResult implicitly converts from LogicalResult (failure $\Rightarrow$ interrupt) — handy for verifier-style walks.

mlir/include/mlir/Support/WalkResult.h:29-55

Mutating during a walk: the rules

The documented guarantee (Operation.h): the callback may **erase the op it is currently visiting** if

- the walk is **post-order** (the default), **or**
- the walk is pre-order **and** you return `WalkResult::skip()` after erasing.

Everything else — erasing *other* not-yet-visited ops, erasing your parent — invalidates the iteration. Undefined behavior, often "works" until it doesn't.

The robust beginner idiom: collect, then mutate.

```
SmallVector<arith::MulIOp> candidates;
getOperation()->walk([&](arith::MulIOp op) { candidates.push_back(op); });

for (arith::MulIOp op : candidates) {
    // ... create replacement, RAUW, op->erase() — no live iterator to break
}
```

Two phases, no aliasing between iteration and mutation. Costs one `SmallVector`; saves one afternoon of debugging.

Live demo: seeing the structure with a real tool

```
build/bin/mlir-opt -test-print-nesting -allow-unregistered-dialect \  
mlir/test/IR/print-ir-nesting.mlir -o /dev/null
```

```
visiting op: 'builtin.module' with 0 operands and 0 results  
1 nested regions:  
  Region with 1 blocks:  
    Block with 0 arguments, 0 successors, and 2 operations  
      visiting op: 'dialect.op1' with 0 operands and 4 results  
      1 attributes:  
        - 'attribute name' : '42 : i32'  
      0 nested regions:  
      visiting op: 'dialect.op2' with 0 operands and 0 results  
      ...  
    Region with 3 blocks:  
      Block with 0 arguments, 2 successors, and 2 operations
```

OpBuilder: a cursor into the IR

To create ops you need two things: an `MLIRContext` and a **place**. `OpBuilder` bundles both — it's a cursor (`Block*` + position) where every newly created op is inserted:

```
OpBuilder b(ctx);                // ⚠ NO insertion point yet!  
b.setInsertionPointToStart(&block); // cursor: start of block  
b.setInsertionPoint(op);          // cursor: right BEFORE op  
b.setInsertionPointAfter(op);     // cursor: right after op  
  
OpBuilder before(op);            // shorthand: constructor == setInsertionPoint(op)  
OpBuilder atEnd = OpBuilder::atBlockEnd(&block);
```

Gotcha #1: `OpBuilder b(op)` inserts **BEFORE** `op` — not after. (Perfect when replacing `op`; surprising otherwise.)

Gotcha #2: `OpBuilder b(ctx)` alone has no insertion point. Creating an op then does **not** crash — the op is built *detached*, inserted nowhere: it silently never appears in your output (and leaks). The docs call this an error; the API won't stop you.

Saving your place: InsertionGuard

Moving the cursor deep into a region and forgetting to move it back is the classic “my ops ended up in the wrong block” bug. RAII to the rescue:

```
{
  OpBuilder::InsertionGuard guard(b); // saves the current insertion point
  b.setInsertionPointToStart(&ifOp.getThenRegion().front());
  arith::ConstantOp::create(b, loc, b.getI32IntegerAttr(42));
} // cursor restored here, automatically
```

- Use it whenever a helper function temporarily redirects a builder it doesn't own.
- Nested guards nest correctly (it's just save/restore).

mlir/include/mlir/IR/Builders.h:350-376

Creating ops: `OpTy::create(builder, loc, ...)`

Every ODS op gets static `create` methods — one per `build(...)` overload you saw in the ODS session:

```
OpBuilder b(op);                // insert before the op we're replacing
Location loc = op.getLoc();      // reuse its location (more in a second)

Value shift = arith::ConstantOp::create(
    b, loc, b.getIntegerAttr(op.getType(), 3));
Value shl = arith::ShLIOp::create(b, loc, op.getLhs(), shift);
```

Heads-up for old code: you will see `builder.create<arith::ShLIOp>(loc, ...)` in blog posts, pre-2025 tutorials, and LLM output. That spelling is **deprecated** (since Oct 2025) — same behavior, but new code must use `OpTy::create(builder, loc, ...)`. Mentally rewrite the old form when you see it.

deprecation: `mlir/include/mlir/IR/Builders.h:506-508`

Constants, attributes, and Location discipline 🕒

`OpBuilder` inherits a bag of factory helpers from `Builder` (types, attributes, locations):

```
IntegerAttr a = b.getI32IntegerAttr(7);          // 7 : i32
IntegerAttr ix = b.getIndexAttr(4);              // 4 : index
Type i32      = b.getI32Type();
IntegerAttr c = b.getIntegerAttr(someType, 3);    // typed constant payload

// A constant op is: attribute payload + arith.constant
Value cst = arith::ConstantOp::create(b, loc, b.getIntegerAttr(ty, 3));
```

Location discipline. Every op carries a `Location` (diagnostics + debug info survive on it):

- When replacing an op, **reuse** `op.getLoc()` for everything you create in its place.
- `b.getUnknownLoc()` is the last resort — an op with unknown location can't point users anywhere when a diagnostic fires.

One-liner to know: `b.createOrFold<OpTy>(loc, ...)` — create the op *and immediately try to fold it*; returns a `Value` that may be a pre-existing constant instead of a new op. (Folding is Session 3's topic.)

Quiz: spot the bug

Goal: rewrite `mul` `%x, c` (`C` a power of two) into `shl`. Does this work?

```
void runOnOperation() override {
  getOperation()->walk([&](arith::MulIOp op) {
    APInt rhs;
    if (!matchPattern(op.getRhs(), m_ConstantInt(&rhs)) || !rhs.isPowerOf2())
      return;
    OpBuilder b(op);
    Value shift = arith::ConstantOp::create(
      b, op.getLoc(), b.getIntegerAttr(op.getType(), rhs.logBase2()));
    Value shl = arith::ShLIOp::create(b, op.getLoc(), op.getLhs(), shift);
    op->erase(); // (A)
    op->replaceAllUsesWith(ValueRange{shl}); // (B)
  });
}
```

Three candidate concerns — which are real bugs?

① The erase/RAUW order (A)/(B) ② erasing `op` *inside* the walk ③ creating new ops *inside* the walk

First use: `matchPattern(v, m_ConstantInt(&rhs))` = "is `v` a constant integer? bind its value into `rhs`" — details on a later slide.

✓ Spot-the-bug answers

① **is the bug** — and it's two bugs in one:

- `erase()` while the `multi`'s result still has uses → fatal error in assert builds:
"operation destroyed but still has uses" (release builds: silent corruption).
- Line (B) then touches the *freed* op — use-after-free.

Fix: swap the two lines. RAUW first, erase second. Always.

② **is legal** — a post-order walk may erase the *currently visited* op (documented guarantee). Erasing *other* ops during the walk would be the bug.

③ **is fine** — new ops inserted before the current op are simply never visited by the ongoing post-order walk.

Still, the model solution uses **collect-then-mutate**: it doesn't depend on walk-order fine print, and it survives refactoring.

Worked example: MulToShift, the real thing

```
void runOnOperation() override {
    // Collect first, mutate after: don't erase while iterating.
    SmallVector<arith::MulIOp> candidates;
    getOperation()->walk([&](arith::MulIOp op) { candidates.push_back(op); });

    for (arith::MulIOp op : candidates) {
        APInt rhs;
        if (!matchPattern(op.getRhs(), m_ConstantInt(&rhs)) || !rhs.isPowerOf2())
            continue;
        OpBuilder b(op); // insertion point: right before `op`
        Value shift = arith::ConstantOp::create(
            b, op.getLoc(), b.getIntegerAttr(op.getType(), rhs.logBase2()));
        Value shl = arith::ShLIOp::create(b, op.getLoc(), op.getLhs(), shift);
        op->replaceAllUsesWith(ValueRange{shl}); // RAUW
        op->erase(); // now safe: use_empty()
    }
}
```

- `matchPattern(value, m_ConstantInt(&rhs))` — "is this value a constant integer? if so, bind it into `rhs`" (an `APInt`). Matches *any* constant-like op, not just `arith.constant`.
- `rhs.isPowerOf2()`, `rhs.logBase2()` — `APInt` does the math.

● Live demo: run it

```
build/bin/mlir-opt mul.mlir --load-pass-plugin=./mul_to_shift.so \
  --pass-pipeline='builtin.module(func.func(mul-to-shift))'
```

Before (mul.mlir)

```
func.func @mul_by_8(%x: i32) -> i32 {
  %c8 = arith.constant 8 : i32
  %r = arith.muli %x, %c8 : i32
  return %r : i32
}

func.func @mul_by_7(%x: i32) -> i32 {
  %c7 = arith.constant 7 : i32
  %r = arith.muli %x, %c7 : i32
  return %r : i32
}
```

After (real output)

```
func.func @mul_by_8(%arg0: i32) -> i32 {
  %c8_i32 = arith.constant 8 : i32
  %c3_i32 = arith.constant 3 : i32
  %0 = arith.shli %arg0, %c3_i32 : i32
  return %0 : i32
}

func.func @mul_by_7(%arg0: i32) -> i32 {
  %c7_i32 = arith.constant 7 : i32
  %0 = arith.muli %arg0, %c7_i32 : i32
  return %0 : i32
}
```

@mul_by_7 untouched (7 isn't a power of two) — but look at @mul_by_8 : **%c8_i32 is now dead.** We never erased it. Who cleans that up? → Session 3. (Spoiler: append ,canonicalize to the pipeline.)

From snippet to pass: what's still missing

Our `runOnOperation` needs a home. A **pass** needs:

- a **name** and a **CLI flag** (`-mul-to-shift`)
- an **anchor**: which op does it run on? (`func.func` for us)
- **registration** so `mlir-opt` / `school-opt` can find it
- plumbing: `clonePass` , options, statistics, ...

You write **none of that by hand**. The modern recipe:

Passes.td	TableGen definition (flag, anchor, docs, options)
mlir-tblgen -gen-pass-decls	
▼	
Passes.h.inc	generated: impl::<Name>Base<Derived> CRTP base class
you derive from it	
▼	
YourPass.cpp	#define GEN_PASS_DEF_<NAME> + runOnOperation()

The smallest real pass: `-strip-debuginfo`

The **entire** implementation, from upstream (license, includes, `using namespace mlir;`, and one inner loop trimmed):

```
namespace mlir {
#define GEN_PASS_DEF_STRIPDEBUGINFOPASS
#include "mlir/Transforms/Passes.h.inc"
} // namespace mlir

namespace {
struct StripDebugInfo : public impl::StripDebugInfoPassBase<StripDebugInfo> {
    void runOnOperation() override;
};
} // namespace

void StripDebugInfo::runOnOperation() {
    auto unknownLoc = UnknownLoc::get(&getContext());
    getOperation()->walk([&](Operation *op) {
        op->setLoc(unknownLoc);
        // ... also resets block-argument locations (three nested loops) ...
    });
}
```

`mlir/lib/Transforms/StripDebugInfo.cpp:14-42`

And its TableGen side — this is *all* of it:

A pass is often just a walk + a library call 🕒

-loop-invariant-code-motion , complete runOnOperation :

```
void LoopInvariantCodeMotion::runOnOperation() {  
    // Walk through all loops in a function in innermost-loop-first order. This  
    // way, we first LICM from the inner loop, and place the ops in  
    // the outer loop, which in turn can be further LICM'ed.  
    getOperation()->walk(  
        [&](LoopLikeOpInterface loopLike) { moveLoopInvariantCode(loopLike); });  
}
```

mlir/lib/Transforms/LoopInvariantCodeMotion.cpp:41-47

- The walk callback filters by **interface** — this pass hoists out of `scf.for` , `affine.for` , *and any future loop op* without knowing them.
- Post-order = innermost loops first — the comment relies on the default walk order you learned 10 slides ago.
- The heavy lifting (`moveLoopInvariantCode`) is a reusable library function.

Your exercise starter: `- school-strength-reduce`

```
// exercises: include/School/SchoolPasses.td (description + statistics elided)
def SchoolStrengthReduce : Pass<"school-strength-reduce", "::mlir::func::FuncOp"> {
  let summary = "Rewrite muli-by-power-of-two into shli (hand-written walk)";
}
```

```
// exercises: lib/School/StrengthReduce.cpp – your Exercise 1 starter (TODO text abridged)
namespace mlir::school {
#define GEN_PASS_DEF_SCHOOLSTRENGTHREDUCE
#include "School/SchoolPasses.h.inc"

namespace {
struct SchoolStrengthReduce
  : public impl::SchoolStrengthReduceBase<SchoolStrengthReduce> {
  void runOnOperation() override {
    // TODO(exercise 1, step 1): Find the candidates (walk + match).
    // TODO(exercise 1, step 2): Build the replacement ops.
    // TODO(exercise 1, step 3): Replace and erase.
  }
};
} // namespace
} // namespace mlir::school
```

The second `Pass<...>` argument **anchors** the pass: it runs on (and gets scheduled per) `func.func`. `getOperation()` is typed accordingly — it returns a `func::FuncOp`.

Pass options and statistics 🕒

Declared in TableGen; materialize as *member variables* of the generated base:

```
// real upstream examples (summary/description fields elided)
def TrivialDeadCodeEliminationPass : Pass<"trivial-dce"> {
  let options = [
    Option<"recursive", "recursive", "bool", /*default=*/"true",
      "Recursively visit nested regions">,
    Option<"removeBlocks", "remove-blocks", "bool", /*default=*/"true",
      "Remove unreachable blocks">];
}
def CSEPass : Pass<"cse"> {
  let statistics = [
    Statistic<"numCSE", "num-cse'd", "Number of operations CSE'd">,
    Statistic<"numDCE", "num-dce'd", "Number of operations DCE'd">];
}
```

mlir/include/mlir/Transforms/Passes.td:103-125, 88-101

In C++: read `recursive` like a bool; assign/increment `numCSE` like an int. On the CLI (📷 captured output):

```
$ mlir-opt in.mlir --pass-pipeline="builtin.module(func.func(cse))" --mlir-pass-statistics
'func.func' Pipeline
  CSEPass
    (S) 2 num-cse'd - Number of operations CSE'd
```

Failing, and staying honest 🕒

Failing: something's broken → `signalPassFailure()`.

```
if (failed(somethingImportant()))  
    return signalPassFailure(); // note the return!
```

- It **sets a bit** — it does not return, throw, or stop your code. The idiom is `return signalPassFailure();`
- The pipeline stops after your pass; `PassManager::run` returns failure; the IR may be left in an invalid state.

Staying honest: after *every* pass, the pass manager runs the **verifier** on the result (`--verify-each` , on by default).

- Your pass must leave IR that verifies — broken dominance, wrong operand types, malformed ops get caught right after *your* pass ran, not three passes later.
- The error names the offending *op* (e.g. `error: 'test.foo' op requires attribute 'attr'`); to see which pass broke it, add `--mlir-print-ir-after-failure` or `--mlir-print-`



Quiz: the cursor was somewhere else 🕒

A colleague "simplifies" the worked example: *"I create the new ops at the end of the block — that spot always exists."* RAUW and erase are in the correct order!

```
getOperation()->walk([&](arith::MulIOp op) {
    APInt c;
    if (!matchPattern(op.getRhs(),
                      m_ConstantInt(&c)) ||
        !c.isPowerOf2())
        return;
    OpBuilder b(
        op->getBlock()->getTerminator());
    Value shift = arith::ConstantOp::create(
        b, op.getLoc(),
        b.getIntegerAttr(op.getType(),
                         c.logBase2()));
    Value shl = arith::ShLIOp::create(
        b, op.getLoc(), op.getLhs(), shift);
    op->replaceAllUsesWith(ValueRange{shl});
    op->erase();
});
```

```
func.func @f(%x: i32) -> i32 {
    %c8 = arith.constant 8 : i32
    %r = arith.muli %x, %c8 : i32
    %s = arith.addi %r, %x : i32
    return %s : i32
}
```

The pass body runs without crashing.

What does `mlir-opt` report?

- ① nothing — clean output
- ② operation destroyed but still has uses
- ③ operand #0 does not dominate this use
- ④ the new `shli` is trivially dead and vanishes

✓ The verifier catches it (③) — dominance 🕒

The block after the rewrite, before verification — reading it top-down shows the bug:

```
func.func @f(%x: i32) -> i32 {  
  %c8 = arith.constant 8 : i32  
  %s = arith.addi %0, %x : i32      // ← now uses %0 ... which is defined below!  
  %c3 = arith.constant 3 : i32  
  %0 = arith.shli %x, %c3 : i32     // ← created "at the end of the block"  
  return %s : i32  
}
```

```
error: operand #0 does not dominate this use  
note: operand defined here (op in the same block)
```

- `replaceAllUsesWith` rewires **every** use — including `%s`, **above** the cursor. Creating the ops "worked"; the SSA graph is broken anyway.
- Your pass completes normally; the **after-pass verifier** rejects the IR and names an op you never touched. (That's "broken dominance" from the previous slide, made concrete.)
- **Rule: choose the insertion point from where the users are — not where it's convenient.** For a 1:1 replacement, `OpBuilder b(op)` — right before the op being replaced

PassManager: a tree of pipelines 🕒

Pipelines are **anchored** and **nested**, mirroring the IR structure:

```
PassManager (anchor: builtin.module)
├─ inline                               ← runs on the module
├─ OpPassManager (func.func)           ← nested pipeline
│   └─ canonicalize                     ← runs on EVERY func.func ...
│       └─ cse                          ← ... potentially in parallel
```

```
mlir::PassManager pm(module.get()->getName());
pm.addPass(mlir::createInlinerPass());           // module-level

mlir::OpPassManager &optPM = pm.nest<mlir::func::FuncOp>();
optPM.addPass(mlir::createCanonicalizerPass());  // function-level
optPM.addPass(mlir::createCSEPass());

if (mlir::failed(pm.run(*module)))
    return 4;
```

adapted from mlir/examples/toy/Ch6/toyc.cpp:149-196

Shorthand for one-offs: `pm.addNestedPass<func::FuncOp>(createCSEPass());`

The nesting gotcha (you *will* hit this today) 🕒

Adding a `func.func`-anchored pass directly to a module-level pipeline is a **hard error**, not an auto-fix:

```
$ mlir-opt in.mlir --pass-pipeline="builtin.module(affine-loop-tile)"
error: Can't add pass 'AffineLoopTiling' restricted to 'func.func' on a
PassManager intended to run on 'builtin.module', did you intend to nest?
```

...and running an anchored pipeline on the wrong op fails too:

```
$ mlir-opt in.mlir --pass-pipeline="func.func(cse)"
error: can't run 'func.func' pass manager on 'builtin.module' op
```

Confusingly: the bare flag `mlir-opt -affine-loop-tile in.mlir` *works* — bare flags use **implicit nesting** and silently build `builtin.module(func.func(affine-loop-tile))`.

See what was actually built: `--dump-pass-pipeline`.

Textual pipelines: the syntax 🕒

```
build/bin/mlir-opt in.mlir \  
  --pass-pipeline="builtin.module(func.func(canonicalize{top-down=true max-iterations=5},cse))"
```

- Starts with the **top-level anchor** (`builtin.module` , or `any` for op-agnostic).
- Parentheses = nesting. **Commas separate passes.**
- Options in braces, **space-separated** `key=value` pairs — *not* commas!

```
$ ... --dump-pass-pipeline  
Pass Manager with 1 passes:  
builtin.module(  
  func.func(  
    canonicalize{ ... max-iterations=5 ... top-down=true},  
    cse  
  )  
)
```

(Real output, trimmed: the dump spells out *all* option values, including defaults you didn't set.)

Quiz: three pipelines

What does each one do — run, or error (and why)?

A

```
mlir-opt in.mlir --pass-pipeline="builtin.module(func.func(canonicalize{top-down=true max-iterations=5},cse))"
```

B

```
mlir-opt in.mlir --pass-pipeline="func.func(cse)"
```

C

```
mlir-opt in.mlir --pass-pipeline="builtin.module(affine-loop-tile)"
```

Bonus: what does plain `mlir-opt -affine-loop-tile in.mlir` do?

✓ Pipeline quiz answers 🕒

	Verdict	Why
A	✓ runs	Full explicit tree; <code>canonicalize</code> + <code>cse</code> on every <code>func.func</code> .
B	✗ error	<code>can't run 'func.func' pass manager on 'builtin.module' op</code> — the string must start at the top-level anchor.
C	✗ error	<code>...restricted to 'func.func' ... did you intend to nest?</code> — explicit pipelines never auto-nest.
bonus	✓ runs	Bare flags use implicit nesting → <code>builtin.module(func.func(affine-loop-tile))</code> .

Rule of thumb: `--pass-pipeline` = **you spell out the whole tree**. Bare flags = `mlir-opt` **guesses the tree**. When surprised: `--dump-pass-pipeline` .

Why anchors matter: threading 🕒

The pass manager runs an anchored pipeline **in parallel over sibling anchor ops** — all `func.func` in a module at once, one thread each (pipeline copies per thread; the whole pipeline runs per function, function by function).

From the `operationPass` class comment — the law:

```
/// Operation passes must not:  
/// - modify any other operations within the parent region, as other threads  
///   may be manipulating them concurrently.  
/// - modify any state within the parent operation, this includes adding  
///   additional operations.
```

`mlir/include/mlir/Pass/Pass.h:354-360`

Practical rules for your pass:

- Touch only IR **nested under** `getOperation()`. Never sibling functions, never the parent module. (Reading ancestors is permitted; mutating them is not.)
- No mutable state across `runOnOperation` calls; no global mutable state. Passes must be

The debugging toolkit (1): watch the IR move

Flag	What it does
<code>--mlir-print-ir-before=cse</code>	dump IR before a specific pass
<code>--mlir-print-ir-after=cse</code>	dump IR after a specific pass
<code>--mlir-print-ir-before-all</code> / <code>--mlir-print-ir-after-all</code>	dump around <i>every</i> pass
<code>--mlir-print-ir-after-change</code>	after-dumps only when the pass changed something
<code>--mlir-print-ir-after-failure</code>	dump only when a pass fails
<code>--mlir-print-ir-tree-dir=/tmp/dumps</code>	one file per pass, in a directory tree

Banner format (real):

```
// -----// IR Dump After CSEPass: cse //----- //
```

The debugging toolkit (2): time, count, trace, reproduce



Flag	What it does
<code>--mlir-timing</code>	wall time per pass (<code>--mlir-timing-display=list tree</code>)
<code>--mlir-pass-statistics</code>	print pass <code>Statistic</code> s (like CSE's <code>num-cse'd</code>)
<code>--debug-only=greedy-rewriter</code>	targeted <code>LLVM_DEBUG</code> tracing (<code>--debug = firehose</code>)
<code>--dump-pass-pipeline</code>	show the pipeline tree that was actually built
<code>--verify-each=false</code>	skip inter-pass verification (perf runs only!)
<code>--mlir-pass-pipeline-crash-reproducer=repro.mlir</code>	on failure/crash: write input IR + pipeline config into one file

● Live demo: `--mlir-print-ir-after-all`

```
build/bin/mlir-opt mul.mlir --load-pass-plugin=./mul_to_shift.so \  
  --pass-pipeline='builtin.module(func.func(mul-to-shift,canonicalize))' \  
  --mlir-print-ir-after-all -o /dev/null
```

```
// -----// IR Dump After {anonymous}::MulToShiftPass: mul-to-shift //----- //  
func.func @mul_by_8(%arg0: i32) -> i32 {  
  %c8_i32 = arith.constant 8 : i32  
  %c3_i32 = arith.constant 3 : i32  
  %0 = arith.shli %arg0, %c3_i32 : i32  
  return %0 : i32  
}  
  
// -----// IR Dump After CanonicalizerPass: canonicalize{...} //----- //  
func.func @mul_by_8(%arg0: i32) -> i32 {  
  %c3_i32 = arith.constant 3 : i32  
  %0 = arith.shli %arg0, %c3_i32 : i32  
  return %0 : i32  
}
```

The dead `%c8_i32` visibly survives our pass and dies in `canonicalize` — the whole story in two dumps.

Exercise 1: -school-strength-reduce

Task: in the `exercises/` project, implement the pass stub: rewrite `arith.muli %x, C` (`C` a power of two) → `arith.shli %x, log2(C)`. Hand-written walk — exactly today's material.

Where: the `// TODO(exercise 1, step N)` markers in `lib/School/StrengthReduce.cpp`; run via `school-opt`. Hints in `exercises/exercise1.md`.

Checkpoints (2–3 covered by the FileCheck test `test/exercise1/strength-reduce.mlir`; 1 verified by eye — `stderr` prints):

1. **Match & print** — `llvm::errs()` every `arith.muli` whose RHS is a constant power of two.
(*walk, matchPattern + m_ConstantInt*)
2. **Rewrite** — build the shift-amount constant + `arith.shli` with `OpTy::create`, RAUW, erase the `mul`.
3. **Robustness** — non-constant RHS, non-power-of-two, several `mul`s per function. No crashes.

Check your progress (from `exercises/`):

Recap: what you can do now

- **Model** — IR is `Operation*` all the way down: `ops` → `regions` → `blocks` → `ops`; `Value` = `OpResult` | `BlockArgument`.
- **Navigate** — parent/child accessors, `getDefiningOp` (null for block args!), `getUsers` / `getUses`.
- **Traverse** — `walk()` with typed callbacks, `WalkResult`, post-order default; collect-then-mutate.
- **Create** — `OpBuilder` insertion points (`OpBuilder(op) = before op`), `OpTy::create(b, loc, ...)`, reuse locations.
- **Rewire** — `replaceAllUsesWith` *then* `erase()`. Always in that order.
- **Package** — `Passes.td` → `GEN_PASS_DEF` → `impl::...Base` → `runOnOperation()`; options & statistics for free.
- **Run** — `--pass-pipeline="builtin.module(func.func(...))"`, nesting rules, parallel-over-functions, and the debug flags.

Cheat sheet: today's API surface

```
// Navigate
op->getParentOp(); op->getBlock();
op->getOperand(0); op->getResults();
v.getDefiningOp<arith::ConstantOp>();
v.getUsers(); v.use_empty(); v.hasOneUse();

// Match constants
APInt c;
matchPattern(v, m_ConstantInt(&c));
c.isPowerOf2(); c.logBase2();

// Rewire (in this order!)
oldOp->replaceAllUsesWith(newOp->getResults());
oldOp->erase();

// Traverse
root->walk([](arith::MulIOp m) { ... });
root->walk([](Operation *o) {
    return WalkResult::interrupt(); });

// Create
OpBuilder b(op); // BEFORE op!
OpBuilder::InsertionGuard g(b);
auto s = arith::ShLIOp::create(b, loc, x, y);

// Pass
struct P : impl::PPassBase<P> {
    void runOnOperation() override; };
return signalPassFailure();
```

```
mlir-opt in.mlir --pass-pipeline="builtin.module(func.func(my-pass{opt=1},cse))" \
--dump-pass-pipeline --mlir-print-ir-after-all --mlir-timing --mlir-pass-statistics
```

Further reading (in your checkout)

- `mlir/docs/Tutorials/UnderstandingTheIRStructure.md` — today's traversal/def-use content, with diagrams; maps 1:1 to `-test-print-nesting` / `-test-print-defuse`.
⚠ Its walk examples call `getFunction()` — that accessor is gone; read it as `getOperation()`.
- `mlir/docs/PassManagement.md` — the authoritative pass-infrastructure reference (anchoring, threading contract, options, statistics, reproducers).
⚠ Some console-output examples predate current behavior (old dump banners, unanchored `-pass-pipeline` strings) — trust the binary over the prose.
- `mlir/docs/Tutorials/MlirOpt.md` — a guided tour of `mlir-opt` flags with runnable examples.
- Source worth reading whole: `mlir/lib/Transforms/StripDebugInfo.cpp`, `LoopInvariantCodeMotion.cpp`, `mlir/test/lib/IR/TestPrintNesting.cpp`.

Next session: stop writing the boring parts

Today you hand-wrote: the traversal, the match, the RAUW, the erase — and left a dead constant behind.

```
// Session 2 preview: the SAME rewrite as a pattern (Exercise 2A's MulByPow2ToShl)
struct MulByPow2ToShl : OpRewritePattern<arith::MulIOp> {
    using OpRewritePattern::OpRewritePattern;
    LogicalResult matchAndRewrite(arith::MulIOp op,
                                   PatternRewriter &rewriter) const override {
        // ... just the match + the replacement. Nothing else.
    }
};
```

A **driver** (`applyPatternsGreedily`) brings: the worklist, re-visiting changed ops, fixpoint iteration, folding, **and dead-code cleanup** — your leftover constant disappears for free.

Session 2: Rewrite Patterns & Dialect Conversion. Bring your Exercise 1 solution — we'll port it in ten lines.